```
    forces = {0 for particle in
particles}                                          #5
    for particle in
particles                                                    #6
      neighbors ← tree.pointsInSphere(particle.poisition,
radius)    #7
      for neighbor in
neighbors                                                  #8
        forces[particle] += computeInteraction(particle,
neighbor)            #9
    for particle in particles
      update(particle, forces[particle])
                  #10
```

#1 Define a function to run the simulation, to which we pass the number of particles and a predicate that becomes true once the simulation is complete (might be based on number of iterations or other conditions, f.i. the system getting into a stable state)

#2 First we need to initialize our `N` particles: depending on the simulation we could desire a random initialization or rather setting a specific configuration.

#3 Run a loop until the simulation completes.

#4 At each step of the simulation, we need to initialize an S-tree (or similar DS) with the current configuration of the system. Since the position of the particles change at each step, we need to update the tree or create a new one every time. We assume the simulation is in a 3D environment, but there are special cases where the dimension of the tuples could be different.

#5 Initialize an array holding, for each particle, the resultant force operating on it. Each entry is a 3D vector (for the assumption made above), holding the vector sum of all